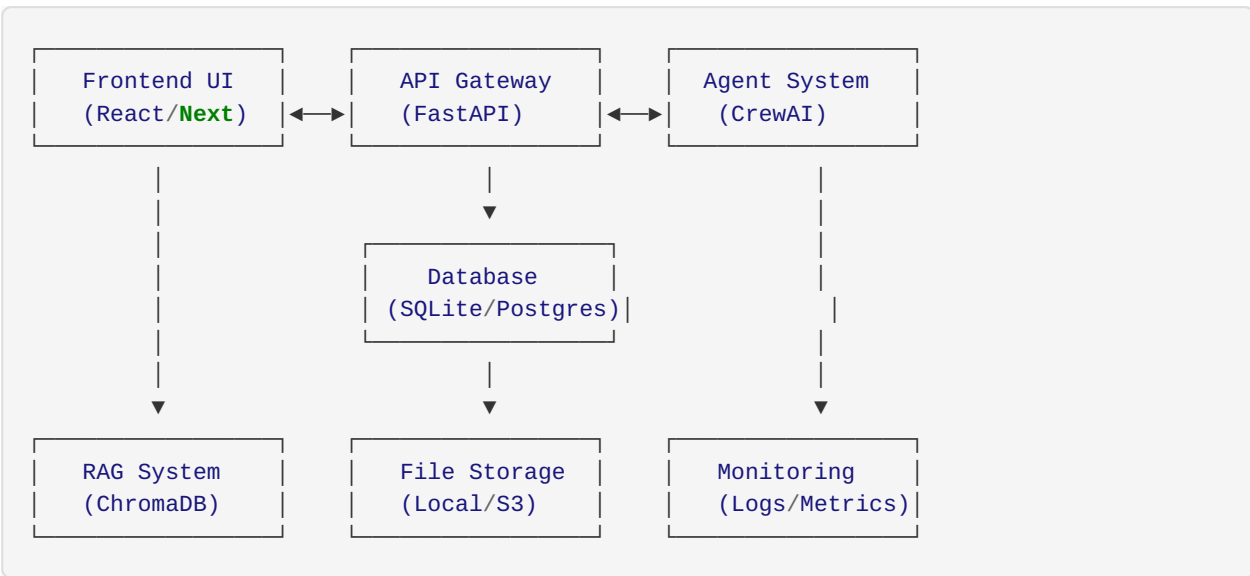


# System Architecture Documentation

## Overview

The Enhanced Agentic AI SDLC System is a comprehensive platform that combines traditional software development lifecycle management with AI-powered agents and retrieval-augmented generation (RAG) capabilities.

## High-Level Architecture



## Component Architecture

### 1. Frontend Layer (UI/)

- **Technology:** React, Next.js, TypeScript
- **Purpose:** User interface for project management, task tracking, and system interaction
- **Components:**
  - Dashboard interfaces
  - Project management views
  - Task tracking components
  - Agent interaction panels
  - Document management UI

### 2. API Layer (API/)

- **Technology:** FastAPI, Python
- **Purpose:** RESTful API for all system operations
- **Features:**
  - CRUD operations for projects, tasks, documents
  - Agent management endpoints
  - Authentication and authorization
  - WebSocket support for real-time updates

- File upload and processing

### 3. Agent System (AGENTS/)

- **Technology:** CrewAI, LangChain
- **Purpose:** AI-powered automation and assistance
- **Agents:**
  - Project Manager Agent
  - Developer Agent
  - QA Engineer Agent
  - Documentation Agent
- **Capabilities:**
  - Task automation
  - Code generation
  - Testing assistance
  - Documentation creation

### 4. RAG System (RAG/)

- **Technology:** ChromaDB, Sentence Transformers
- **Purpose:** Knowledge management and retrieval
- **Features:**
  - Document embedding and storage
  - Semantic search
  - Context-aware responses
  - Knowledge base management

### 5. Data Layer (MODELS/)

- **Technology:** SQLAlchemy, SQLite/PostgreSQL
- **Purpose:** Data persistence and management
- **Models:**
  - Projects
  - Tasks
  - Documents
  - Agents
  - Users (future)

### 6. Business Logic (SERVICES/)

- **Technology:** Python
- **Purpose:** Core business logic and workflows
- **Services:**
  - Project management
  - Task orchestration
  - Document processing
  - Agent coordination

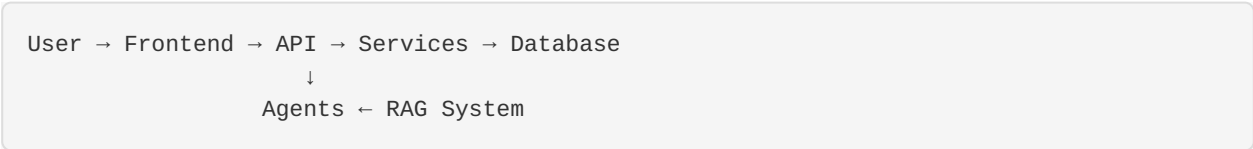
### 7. Utilities (UTILS/)

- **Technology:** Python

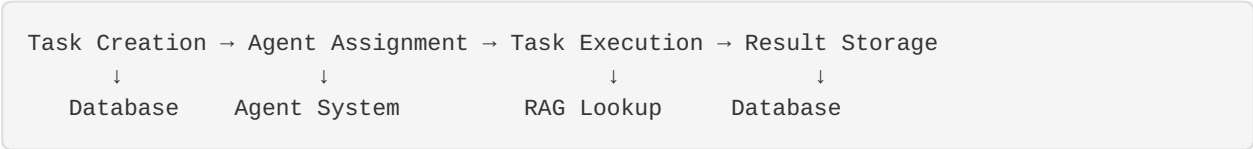
- **Purpose:** Helper functions and common utilities
- **Features:**
  - File validation
  - Configuration management
  - Logging utilities
  - Data processing helpers

## Data Flow

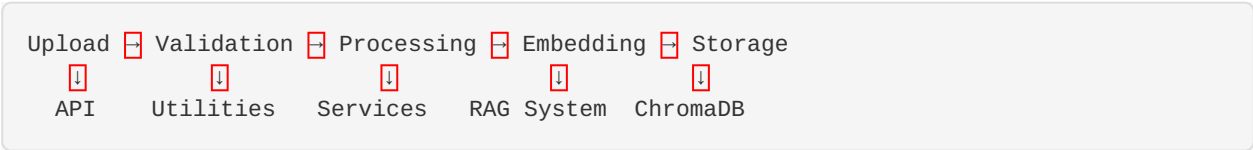
### 1. User Request Flow



### 2. Agent Workflow



### 3. Document Processing



## Technology Stack

### Backend

- **Framework:** FastAPI
- **Database:** SQLAlchemy with SQLite/PostgreSQL
- **AI/ML:** OpenAI, LangChain, CrewAI
- **Vector DB:** ChromaDB
- **Embeddings:** Sentence Transformers

### Frontend

- **Framework:** React/Next.js
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **State Management:** React Hooks/Context

### Infrastructure

- **Containerization:** Docker
- **Orchestration:** Docker Compose
- **Web Server:** Uvicorn
- **Reverse Proxy:** Nginx (production)

# Security Architecture

---

## Authentication & Authorization

- JWT-based authentication
- Role-based access control (RBAC)
- API key management for external services

## Data Security

- Environment variable management
- Secure file upload handling
- Database connection encryption
- Input validation and sanitization

## API Security

- CORS configuration
- Rate limiting
- Request validation
- Error handling without information leakage

# Scalability Considerations

---

## Horizontal Scaling

- Stateless API design
- Database connection pooling
- Microservice-ready architecture
- Load balancer compatibility

## Performance Optimization

- Async/await patterns
- Database query optimization
- Caching strategies (Redis)
- Background task processing

## Resource Management

- Memory-efficient vector operations
- Streaming for large file processing
- Connection pooling
- Resource cleanup

# Monitoring & Observability

---

## Logging

- Structured logging with JSON format
- Log levels (DEBUG, INFO, WARNING, ERROR)
- Request/response logging
- Agent activity logging

## Metrics

- API response times
- Database query performance
- Agent execution metrics
- System resource usage

## Health Checks

- API endpoint health
- Database connectivity
- External service availability
- Agent system status

## Integration Points

---

### External APIs

- OpenAI API for language models
- Third-party authentication providers
- File storage services (S3, etc.)
- Notification services

### Internal Integrations

- Agent-to-Agent communication
- RAG system queries
- Database transactions
- File system operations

## Deployment Architecture

---

### Development

Local Machine

- ☐ SQLite Database
- ☐ Local File Storage
- ☐ Development Server
- ☐ Hot Reload

### Production

Load Balancer

- ├ API Servers (Multiple)
- ├ PostgreSQL Cluster
- ├ Redis Cache
- ├ File Storage (S3)
- └ Monitoring Stack

## Future Enhancements

---

### Planned Features

- Real-time collaboration
- Advanced analytics dashboard
- Multi-tenant support
- Plugin architecture
- Mobile applications

### Scalability Improvements

- Kubernetes deployment
- Microservice decomposition
- Event-driven architecture
- Distributed caching

### AI Enhancements

- Custom model fine-tuning
- Advanced agent capabilities
- Improved RAG performance
- Multi-modal support

---

This architecture provides a solid foundation for a scalable, maintainable, and extensible SDLC system with AI capabilities.